

Practice 3 - Backend

Account Transfer System with Balance Validation in Node.js

Objective: Learn how to implement a secure money transfer API in Node.js and MongoDB without using database transactions. This helps you understand dependent multi-document updates, balance validation, and proper error handling.

Concept Overview:

In financial systems, fund transfers require consistent updates to multiple user accounts. This exercise teaches logical validation and sequential updates to ensure accuracy without database transactions.

Steps / Procedure:

Step 1: Initialize Project

```
mkdir account-transfer-system
cd account-transfer-system
npm init -y
npm install express mongoose body-parser
```

Step 2: Create server.js

```
const express = require('express'); const
mongoose = require('mongoose'); const
bodyParser = require('body-parser'); const
app = express(); const PORT = 3000;

app.use(bodyParser.json());

mongoose.connect('mongodb://localhost:27017/bankDB', {
  useNewUrlParser: true,   useUnifiedTopology: true });

const userSchema = new mongoose.Schema({
  name: String,   balance: Number });

const User = mongoose.model('User', userSchema);

app.post('/create-users', async (req, res) => {
  try {      await User.deleteMany({});      const
  users = await User.insertMany([      { name:
  'Alice', balance: 1000 },
    { name: 'Bob', balance: 500 }      ]);
  res.status(201).json({ message: 'Users created', users });
} catch (err) {      res.status(500).json({ message: 'Error
creating users' });      });

app.post('/transfer', async (req, res) => {  try {
const { fromUserId, toUserId, amount } = req.body;
const sender = await User.findById(fromUserId);
const receiver = await User.findById(toUserId);
```

```

    if (!sender || !receiver) {      return
    res.status(404).json({ message: 'User not found' });    }

    if (sender.balance < amount) {    return res.status(400).json({
message: 'Insufficient balance' });    }

    sender.balance -= amount;
    receiver.balance += amount;

    await sender.save();
    await receiver.save();

    res.status(200).json({      message: `Transferred ${amount} from
${sender.name} to ${receiver.name}`,      senderBalance: sender.balance,
receiverBalance: receiver.balance    });    } catch (error) {
res.status(500).json({ message: 'Transfer failed', error: error.message });    }
});

app.listen(PORT, () => {    console.log(`Server running on
http://localhost:${PORT}`); });

```

Expected Output:

The image displays three screenshots of a REST client interface, likely Postman, showing the results of API calls.

First Screenshot: A POST request to `http://localhost:3000/create-users` is shown. The response is a 201 status code, labeled "Created". The JSON body contains a message "Users created" and an array of two user objects. The first user is Alice with a balance of 1000 and a unique ID. The second user is Bob with a balance of 500 and a unique ID.

Second Screenshot: A POST request to `http://localhost:3000/transfer` is shown. The request body includes the IDs of Alice and Bob, and a transfer amount of 150. The response is a 200 status code, labeled "OK". The JSON body contains a message "Transferred \$150 from Alice to Bob", the updated sender balance of 850, and the updated receiver balance of 650.

Third Screenshot: A POST request to `http://localhost:3000/transfer` is shown. The request body is identical to the second one, but with a transfer amount of 900. The response is a 400 status code, labeled "Bad Request". The JSON body contains a message "Insufficient balance", indicating that the transfer amount exceeds the sender's current balance.

Result:

The system successfully transfers funds between users with proper validation and error handling. Logical checks ensure consistent data even without transactions.